Singleton Pattern

Idea – "a class which is instantiated only once"

– e.g. – db conn, redis conn, logger

Singleton decorator

```
def singleton(class_):
    instances = {}

    def get_instances(*args, **kwargs):
        if class_ not in instances:
            instances[class_] = class_(*args, **kwargs)

        return instances[class_]

    return get_instances()
```

```
@singleton
class DataBase:
    def __init__(self):
        pass

if name == '__main__':
    d1 = DataBase()
    d2 = DataBase()
```

d1 == d2 ⇒ True

Singleton metadata

```
class Singleton(type):
    _instances = {}

    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super(Singleton, cls).
                                  __call__(*args, **kwargs)
        return cls._instances[cls]

class Database(metaclass=Singleton)
    def __init__(self):
        print("Loading Database")

if __name__ == "__main__":
    d1 = Database()  # loading database prints
    d2 = Database()  # once
    print(d1 == d2)  # True
```

Monostate

also known as Borg idiom, is an alternative to the traditional Singleton pattern.
While a Singleton restricts object creation to a single instance, Monostate allows multiple instances but forces them to share the same state through static variables